AIRGUARD INDIA 2.0

Live data ingestion — implementation

DATE	08 October 2026
SOURCE	docs/airguard2/live_data_implementation.md
REPOSITORY	https://github.com/nishaids/airguard-india

Phases 2 and 3 of the AirGuard 2.0 roadmap: data access, and the foundation for live ingestion. This document describes what was built, what has been proven to work, and what has not yet been run.

Nothing here is connected to the forecasting models or to the deployed application. The 1.0 site is unchanged.

Status on 8 October 2026

ITEM	STATE
Connectors for four sources, validation, provenance, storage layer, orchestrator, tests	**Built**
Unit tests (59, all providers mocked)	**Passing**
Open-Meteo weather forecast, live	**Verified end to end**: 804 future forecast hours for 12 cities retrieved, validated, written to PostgreSQL and read back; a second run wrote nothing
OpenAQ, live	**Not run.** The service is reachable and answers 401 without a key; no key was configured on the machine
NASA FIRMS, live	**Not run.** The service is reachable; no key was configured on the machine
CPCB / data.gov.in, live	**Not run, and the host is unreachable.** `api.data.gov.in` refused TCP connections on ports 443 and 80 from the development network on 7 and 8 October 2026, independent of any key
Database migration	**Verified on a local PostgreSQL 17** (a throwaway Docker container). It applied cleanly and a second application was a no-op. **Not yet applied to Supabase**: no connection string was configured
Writing to PostgreSQL and reading back	**Verified locally** for live weather. The station, observation, fire, flag and rejection paths were verified with the test fixtures' provider responses, not live data. Every second pass wrote nothing
Supabase specifically (pooler, TLS, its roles)	**Not run**
Scheduled, unattended ingestion	**Not built**

The acceptance tests that need credentials run with one command once `.env` exists; see *Commands*.

Running against a real database, rather than the in-memory stand-in the unit tests use, found three defects, all corrected:

- A second weather run stored every forecast hour again, because a forecast was identified by the instant it was fetched. It is now identified by the model run.

- The table that records applied migrations was created without row-level security.
- When one monitor arrived from both sources, the rule that picks a copy for modelling changed the official reading after it had been saved, so the change never reached the database, and it counted rejected readings as held.

Providers

SOURCE LABEL	KIND	KEY	WHAT IT IS
`CPCB/data.gov.in`	**measured**, official	`DATA_GOV_API_KEY`	The Central Pollution Control Board's real-time feed on the Government of India open-data platform
`OpenAQ`	**measured**, aggregator	`OPENAQ_API_KEY`	Station readings collected by OpenAQ from government networks and other monitors
`NASA FIRMS`	**satellite fire detection**	`NASA_FIRMS_MAP_KEY`	VIIRS active-fire hotspots from NOAA-20 and NOAA-21
`Open-Meteo`	**weather forecast**	none	Forecast meteorology from national weather models, redistributed by Open-Meteo
`CAMS/Open-Meteo`	**modelled**	none	Registered in the database so that modelled air quality can only ever be stored as modelled. No connector yet

Measured, modelled, and fire detections are never mixed

Every record carries a `source` and a `source_type`, and the database constrains them.

- A **measured** value came from an instrument at a station. Only `CPCB/data.gov.in` and `OpenAQ` produce these.
- A **modelled** value is a model's estimate. The observation table accepts the label, but no modelled record is ingested yet, and the modelling view excludes them.
- A **fire detection** is not air quality. It lives in its own table whose `source_type` column can hold only `satellite_fire_detection`.

If the official feed cannot be reached, the run for that source fails with the words **"Official measured source unavailable."** and its reason. No other source is relabelled to stand in for it. OpenAQ continues to be reported as OpenAQ even for a station whose upstream provider is CPCB.

Credentials

Secrets are read from the process environment only. For local use, copy `.env.example` to `.env` and fill it in; `.env` is ignored by Git and is loaded into the environment at start-up without overriding variables that are already set.

- No secret is written to source, logs, reports, the database or the frontend. There is no `NEXT_PUBLIC_` variant of any of them.
- Two providers put the key where a careless log line would print it: data.gov.in in the query string and FIRMS in the URL path. The HTTP client never logs a URL, every error message passes through a redaction function before it is raised, and a logging filter redacts once more as a last line of defence. Tests assert that a key embedded in a provider's error text does not reach the log or the printed report.
- A `DATABASE_URL` that still contains placeholder text such as `PROJECT_REF` is treated as unconfigured and reported as such.

Canonical schema

Defined in `src/ingestion/schemas.py`. Each provider's dialect is translated into these types; what the provider said is kept beside what AirGuard takes it to mean.

TYPE	IDENTITY (WHAT MAKES INGESTION IDEMPOTENT)	KEY FIELDS
`Station`	source, provider's station id	name, city, state, coordinates, timezone, owner, upstream provider, link to the same monitor in another source
`AirQualityObservation`	source, station id, pollutant, averaging period, observed time	raw value, raw unit, raw minimum and maximum, value kind, normalised value and unit, observed and retrieved times (UTC), local timezone, quality state, flags, freshness, provenance
`WeatherForecast`	source, model, location, valid time, **model run**	issue time, first retrieval time, lead time, seven variables with units
`FireDetection`	source, hash of product, position, time and satellite	satellite, instrument, acquisition time, confidence, fire radiative power, brightness
`DataQualityFlag`	—	code, severity (info, warning, reject), message, field
`IngestionRun`	—	source, start and end, status, counts retrieved, accepted, flagged, rejected, duplicate, written, redacted error

All timestamps are timezone-aware and stored in UTC. A naive datetime is refused rather than assumed. The CPCB feed reports Indian Standard Time without saying so; it is converted, and the original text is kept in provenance.

Why a weather forecast's identity includes the model run

A model predicting tomorrow may use tomorrow's forecast weather, because that exists today. It may not use tomorrow's observed weather. If a stored forecast were overwritten when a newer one arrived, the archive would drift toward observations and a model trained on it would test better than it can ever perform.

So a later model run's forecast for the same valid hour is a **new row**. Forecast rows are never updated.

Fetching the same model run again is the same row, and adds nothing. The copy kept is the first one, so its `retrieved_at` is the earliest moment AirGuard is known to have held that forecast. To reconstruct what was known at a past issue time, read, for each valid hour, the row with the latest `retrieved_at` not after it.

`issued_at` is the weather model's own run-initialisation time, taken from Open-Meteo's model metadata. When the provider publishes none it is left empty rather than guessed, and the edition is identified by the hour of retrieval instead; the column `vintage_at` holds whichever of the two was used.

Data-quality checks

`src/ingestion/validation.py`. The rule is to keep what can be kept: a reading that looks wrong but could be real is stored with a flag; only a record that cannot be used at all is rejected, and the rejection is itself stored with its reason and an excerpt of the record.

CODE	SEVERITY	MEANING
`MISSING_STATION_ID`	reject	The record names no station
`MISSING_TIMESTAMP`	reject	No usable measurement time
`MISSING_POLLUTANT`	reject	The pollutant is not recognised
`UNPARSABLE_VALUE`	reject	The value is not a number (`NA` , empty, text)
`NEGATIVE_VALUE`	reject	A negative concentration
`FUTURE_TIMESTAMP`	reject	Measurement time later than now plus a tolerance (30 minutes by default)
`DUPLICATE_IN_BATCH`	reject	Same station, pollutant and time twice in one batch; counted separately as a duplicate

CODE	SEVERITY	MEANING
`MALFORMED_RECORD`	reject	The provider's record is not the expected shape
`UNIT_UNSPECIFIED`	warning	The provider gives no unit; stored raw, not normalised
`UNSUPPORTED_UNIT`	warning	The unit is not one that can be converted for this pollutant
`IMPLAUSIBLE_VALUE`	warning	Above the plausibility ceiling already used by the 1.0 cleaning stage
`COORDINATES_MISSING`, `COORDINATES_INVALID`, `COORDINATES_OUTSIDE_INDIA`	warning	Position absent, impossible, or outside the India bounding box
`STALE`, `UNAVAILABLE`	warning	Older than the stale or unavailable threshold
`DELAYED`, `UNIT_CONVERTED`, `SAME_MONITOR_OTHER_SOURCE`	info	Recorded for the audit trail; the record stays `ok`

Provider and API failures are not record flags; they fail the run for that source and are stored on the run.

Staleness

STATE	DEFAULT RULE	SETTING
fresh	up to 90 minutes old	`AIRGUARD_FRESH_MINUTES`
delayed	up to 180 minutes old	`AIRGUARD_STALE_MINUTES`
stale	up to 24 hours old	`AIRGUARD_UNAVAILABLE_HOURS`
unavailable	older, or no reading at all	—

These are product decisions. Three hours is the blueprint's proposed limit for showing a reading as current; ninety minutes allows for an hourly feed published with some delay. Neither has been checked against how late these feeds actually run, which is one of the first things real ingestion will show.

Unit normalisation

`src/ingestion/units.py`. AirGuard's units are those already in `config.POLLUTANT_UNITS`: micrograms per cubic metre for particulates and most gases, **milligrams per cubic metre for carbon monoxide**.

FROM	TO	RULE
µg/m³ ↔ mg/m³	either	Exact, factor of 1,000
ppb	µg/m³	`ppb × M ÷ 24.45`
ppm	mg/m³	`ppm × M ÷ 24.45`
ppb or ppm for particulate matter	—	Refused: particles have no molar mass
missing or unknown unit	—	Refused: the raw value is kept and flagged

`M` is the molar mass and 24.45 L/mol is the molar volume of an ideal gas at 25 °C and 1 atmosphere. Ambient temperature changes the result by a few per cent, so every converted record states the assumption in its provenance and carries a `UNIT_CONVERTED` flag. The raw value and raw unit are always stored unchanged beside the normalised ones.

Carbon monoxide is the case most likely to go wrong: a provider reporting 1,400 µg/m³ and another reporting 1.4 mg/m³ mean the same thing, and a test covers exactly that.

Recognising the same monitor from two sources

`src/ingestion/matching.py`. OpenAQ republishes many government stations, so the same instrument can arrive twice.

Two readings are not treated as duplicates because they share a city. Stations are linked only on evidence: within 300 metres of each other (`AIRGUARD_STATION_MATCH_M`), or within two kilometres with near-identical names. The link stores its method and distance. Both records are kept with their own source and provenance; neither is merged, deleted or relabelled.

What changes is which copy a model may read. If both copies have a normalised concentration, the official one is used and the aggregator's is stored with a flag and excluded. If the official copy could not be normalised it cannot be modelled, so the aggregator's copy stays usable and records that an official copy exists. A rejected reading counts for nothing. The official reading is never altered by this rule. The link is stored on the station, and the view `airguard.observations_for_modelling` returns one reading per physical monitor, pollutant and hour whatever order the sources arrived in.

This matcher is preliminary. It has not been checked against a hand-verified list of station pairs, and it has not yet seen real data from both sources at once. Against PostgreSQL it has been exercised only with the test fixtures.

Database

`db/migrations/001_live_ingestion.sql`, applied with `python -m src.ingestion.db migrate`. Everything is in a schema of its own, `airguard`. On Supabase the generated REST API serves only the `public` schema, so these tables cannot be reached with the project's anonymous key; row-level security is also enabled on every table with no policies, as a second barrier. That reasoning about Supabase is from its documentation; it has not been tested against a Supabase project.

TABLE	HOLDS	UNIQUENESS
`source_registry`	The five sources, their kind, whether official, attribution	source id
`stations`	One row per source station, with an optional link to the same monitor elsewhere	source, station id
`air_quality_observations`	Raw and normalised values, times, quality state, provenance	source, station, pollutant, averaging period, observed time
`weather_forecasts`	One row per location, model, valid hour and model run	those four plus coordinates
`fire_detections`	Hotspots	source, record hash
`data_quality_flags`	Flags on stored rows, and the reasons for rejected ones	per row and code
`ingestion_runs`	One row per source per invocation	—
`observations_for_modelling` (view)	One usable measured reading per monitor, pollutant and hour	—

Every insert is `ON CONFLICT DO NOTHING` against these keys, so running ingestion twice writes the data once. There are no tables about people or health in this phase.

Commands

```
python -m src.ingestion.run --all --dry-run            retrieve and validate everything, write nothing
python -m src.ingestion.run --all --dry-run --limit 3  the same, small, for a first check
python -m src.ingestion.run --source weather --dry-run one source
python -m src.ingestion.db migrate                     create the schema
python -m src.ingestion.run --all                      ingest into the database
python -m src.ingestion.db status                      row counts and the latest runs
python -m pytest tests/test_ingestion.py -q            the tests
```

`--limit` means records for CPCB and FIRMS, and locations for OpenAQ and weather. Each source reports `SUCCESS`, `PARTIAL` or `FAILED` on its own; one failing does not stop the others, and the exit code is non-zero if any failed. The output is counts, times, a status and a few sample records, with nothing secret in it.

Reliability

`src/ingestion/http.py`: a 30-second timeout; up to three retries with exponential back-off capped at 30 seconds; retries on connection failure, timeout, 429 and 5xx, and not on other 4xx; `Retry-After` honoured; a response that is not valid JSON, or a FIRMS reply that is not CSV, treated as a failure rather than parsed.

OpenAQ's free tier allows 60 requests a minute and 2,000 an hour and warns that repeated breaches can lead to a ban. The connector reads the remaining-quota header after every response and pauses before the window is spent. It polls only stations that have reported in the last seven days.

Source limitations and open questions

CPCB / data.gov.in

- *Unreachable from the development network.* The API host resolves but refuses connections. Whether this is the network, the service, or a restriction on where requests may come from is not known. It should be tried from a second network before anything else is concluded.
- *What the numbers are is not established.* The feed gives a minimum, maximum and average per station and pollutant. In the releases seen it publishes no unit, and its title describes an index. The connector therefore does **not** assume concentrations: values are stored raw with `value_kind = unknown` and flagged `UNIT_UNSPECIFIED`, which also keeps them out of the modelling view. Whether they are sub-indices or concentrations, and over what window, must be settled by comparing one station against the CPCB portal. Until then this feed can show that a station is reporting, but it cannot feed a model.
- *The resource identifier may change.* Every run reads the dataset's own metadata and refuses anything that is not an air-quality dataset with the expected fields, so a retired identifier fails loudly.
- The feed has no station identifier; one is derived from state, city and station name.

OpenAQ

- An aggregator, with its own delay behind each upstream network. OpenAQ states that "latest" is the last value in a sensor's series, which is not always the most recently ingested.
- Coverage is discovered within 25 km of each of twelve city centres. Stations elsewhere are not collected yet.
- The response shapes are implemented from OpenAQ's documentation and exercised only through mocks. The first real run is the test of that.

NASA FIRMS

- A hotspot is a thermal anomaly, not a measure of smoke or of air quality. Detections are missed under cloud and between overpasses.
- The India bounding box is a rectangle and includes parts of neighbouring countries. That is deliberate — smoke crosses borders — but it is not "fires in India".
- The CSV column names are implemented from the published format and exercised only through mocks.

Open-Meteo

- A third-party redistribution, not an official Indian meteorological source. Its free tier is for non-commercial use.
- `gfs_seamless` blends more than one GFS product; the recorded issue time is the 0.13° run's, noted as such on each row.
- Forecasts are stored for city-centre reference points, not for stations.
- Boundary-layer height can be missing for some hours; those rows are stored and flagged.

What remains before real-time forecasting

1. Put credentials in `.env`; run the dry run for the three keyed sources; apply the migration to Supabase; run a real ingestion there and confirm a second run writes nothing.
2. Establish what the CPCB feed's numbers mean, or settle on OpenAQ as the measured source.
3. Run ingestion unattended on a schedule, and watch it for a week.
4. Validate the station matcher against real pairs.
5. Backfill history, then rebuild the training set from point-in-time snapshots. None of this ingestion feeds a model yet.